Frosch Bottle Inspection

Technical Model Handoff

Dataset · Detection · Segmentation · Geometry · OCR · Tracking · Classification · TensorRT · Benchmarking

Consolidated Handbook Evidence Record

Prepared against the xis.ai AI Departmental Handbook v1.0

EVALUATION HANDOFF · Version 1.0 · September 2026

Field	Details
Project	Frosch Bottle Inspection
Department	AI Department, xis.ai
System boundary	Full-frame multi-class detection + instance segmentation + geometry (tilt / H-V centricity) + capacity OCR + multi-frame tracking → GOOD / DEFECTIVE / INCOMPLETE
Detector	RF-DETR Medium
Segmenter	RF-DETR Seg Medium
OCR	PaddleOCR (GPU)
Runtime backend	Native TensorRT (dedicated CUDA streams)
Dataset	frosch-bottle-5-ypv4gi
Primary status	Evaluation complete in part; release gates remain open
Confidentiality	Internal use only

***Release status.** This package consolidates the current pipeline architecture, recorded detection evidence, sample operational results, runtime optimisations and open handbook gates. It is **not** a production approval. Missing gates are listed in §8 and must be closed before release.*

Contents

Document purpose. This report consolidates the Frosch live-inspection pipeline into one handbook-aligned handoff. Architecture, thresholds, sample results, runtime optimisations and open gates are preserved. Details that were not recorded (exact training hyperparameters, SHA-256 hashes of every checkpoint, formal sealed splits, independent measurement validation) are left explicit rather than invented.

Interpretation rule. Sample bottle tables and the provided detection confusion matrix are evaluation evidence only. They do not by themselves constitute production acceptance. GOOD / DEFECTIVE / INCOMPLETE outcomes depend on the rule set in §5 and must be judged against a signed objective (A02) before release.

§1 Executive Handoff Summary

1.1 System Purpose and Boundary

Frosch is a live computer-vision inspection system for Frosch bottles (100 ml, 300 ml, 500 ml). A camera (or folder of frames) supplies images. The system detects bottle, label, capacity marking, damage, bump and scratch; segments bottle and label masks; measures tilt and label centricity; reads capacity via OCR; confirms defects only when they overlap the bottle mask; tracks each bottle across frames; and outputs **GOOD**, **DEFECTIVE** or **INCOMPLETE**, plus annotated/clean crops, CSV, JSON and optional video.

Pipeline boundary: Camera / frame folder → Detection (RF-DETR Medium, TensorRT) + Segmentation (RF-DETR Seg Medium, TensorRT) → OCR · Orientation · H/V centricity · Mask-validated defects → Multi-frame tracking + temporal aggregation → GOOD / DEFECTIVE / INCOMPLETE → Image / CSV / JSON / video outputs.

Detector, segmenter, OCR and rule logic are separate components. They must be accepted together as one pipeline before production use.

1.2 Current Model Position

Component	Artifact	Recorded evidence	Current standing
Detector	RF-DETR Medium → TensorRT	Confusion matrix (IoU ≥ 0.50); per-class thresholds	Strong detection evidence; release gates open
Segmenter	RF-DETR Seg Medium → TensorRT	Masks for orientation, centroid, defect-overlap	Required for accurate tilt and defect validation
OCR	PaddleOCR (GPU)	Capacity ∈ {100,300,500}; min conf 0.18; throttled	Operational; not formal standalone acceptance
Decision logic	Rule-based	Sample runs for 100 / 300 / 500 ml	Evaluation samples only
Runtime	Native TensorRT + CUDA streams	Full pipeline ~18.7 FPS; inference ~74.7 FPS (sample)	Optimisation evidence present; soak still required

1.3 What Is Already Documented

- System architecture and code split (inference.py / live_inference.py)
- Per-class confidence thresholds, orientation limit, H/V rules, defect-overlap rule
- Capacity-specific expected vertical offsets
- Tracking, trigger-line and missing-frame finalisation
- Sample operational tables for 100 / 300 / 500 ml
- Detection confusion matrix (IoU ≥ 0.50)
- Runtime optimisations (TensorRT, dedicated streams, pinned memory, OCR throttle, async save)
- Supporting docs: README, camerasetup, results_saving, results, optimization

1.4 Release Position

NO RELEASE CANDIDATE. *The current package is suitable for evaluation, reproducibility and integration testing. It does not yet contain a signed customer objective, formal annotation schema / QA record, sealed dataset version with independent validation, soak / recovery evidence, full fallback audit or final approval.*

§2 Dataset, Labels and Split Lineage

2.1 Source Dataset

Dataset field	Recorded value
Dataset ID	frosch-bottle-5-ypv4gi
Domain	Frosch bottle inspection (100 / 300 / 500 ml)
Annotation type	Multi-class detection (+ segmentation masks for bottle / label in operational pipeline)
Primary classes	bottle, label, capacity, damage, bump, scratch

Exact total image count, formal train / val / test counts and SHA-256 of the sealed export are **not** recorded in the materials supplied for this handoff and must be added under A03 / A06.

2.2 Classes and Annotation Scope

Class	Role
bottle	Primary object; drives tracking and geometry
label	Used for H/V centricity

Class	Role
capacity	Region for OCR
damage	Defect candidate (must pass mask overlap)
bump	Defect candidate (must pass mask overlap)
scratch	Present in model vocabulary; operational confirmation focuses on damage / bump

2.3 Classification / Decision Labels

Final bottle status is **not** a direct model class. It is produced by rule logic:

Status	Meaning
GOOD	Orientation PASS, H PASS, V PASS, no confirmed defects
DEFECTIVE	Any of orientation / H / V FAIL, or confirmed damage / bump
INCOMPLETE	One or more required measurements still Pending

Important: Missing data is treated as Pending → INCOMPLETE, not as FAIL.

2.4 Dataset Lineage Note

Operational code references detection checkpoint path `runs/frosch_medium/checkpoint_best_regular.pth`, segmentation checkpoint `runs/frosch_seg_medium/checkpoint_best_total.pth`, and runtime engines `output/rfdetr-medium.trt` / `output/rfdetr-seg-medium.trt`. Formal dataset-version ID, split repair log and parent hashes remain open (A06).

§3 Object Detection — RF-DETR Medium

3.1 Role and Artifact Traceability

RF-DETR Medium performs multi-class detection on each full frame. Outputs feed tracking, OCR crops, defect candidates and association with segmentation masks.

Field	Recorded value
Model family	RF-DETR Medium
Runtime form	Native TensorRT engine
Engine path	output/rfdetr-medium.trt
Training checkpoint (lineage)	runs/frosch_medium/checkpoint_best_regular.pth

Exact training run ID, epoch count, seed, environment lock and checkpoint SHA-256 were not supplied for this handoff and remain required under A07.

3.2 Recorded Detection Performance

Confusion matrix provided for evaluation at **IoU** ≥ **0.50** (rows = ground truth, columns = predicted; includes a “missed” column):

Ground truth	bottle	bump	capacity	damage	label	scratch	missed
bottle	302	0	0	0	0	0	2
bump	3	68	4	10	3	2	29
capacity	0	0	273	0	0	1	1
damage	0	0	2	41	0	0	2
label	0	10	15	4	301	1	2
scratch	1	0	0	6	1	14	22

Observations (evaluation evidence only): bottle, capacity and label show strong diagonal counts. bump and scratch have higher miss and confusion rates. damage has smaller support; misses are visible. No formal mAP@0.50:0.95 or per-capacity P/R/F1 table was supplied beyond this matrix.

3.3 Operational Confidence Thresholds

Class	Confidence threshold
bottle	0.70
label	0.35
capacity	0.35
bump	0.50
damage	0.30
scratch	0.30

Higher threshold on bottle reduces false tracks. Lower thresholds on defects reduce the chance of missing real damage/bump, with mask-overlap acting as a second filter.

3.4 TensorRT Contract

Item	Contract
Engine	output/rfdetr-medium.trt
Execution	Dedicated CUDA stream per engine
Input handling	Shared pinned-host RGB staging → async H2D → preprocess on stream → execute_async_v3 → async D2H
Decode	dets + labels → sigmoid scores, top-k, score filter, xyxy in frame coordinates
Batch	Static batch 1 (dynamic shapes not used in current runner)

§4 Instance Segmentation — RF-DETR Seg Medium

4.1 Role and Why a Separate Segmenter

A second RF-DETR Seg Medium model produces instance masks. Bounding boxes alone are insufficient for accurate bottle orientation (PCA on mask pixels), reliable centroids for centricity, and defect validation (overlap of defect box with bottle mask).

Field	Recorded value
Model family	RF-DETR Seg Medium

Field	Recorded value
Runtime form	Native TensorRT engine
Engine path	output/rfdetr-seg-medium.trt
Training checkpoint (lineage)	runs/frosch_seg_medium/checkpoint_best_total.pth
Segmentation score floor	0.30 (further filtered by per-class rules where applicable)

4.2 Mask Usage in Downstream Logic

- Select best bottle mask by class name + confidence + IoU with detection box.
- Resize / constrain mask to frame and bottle box.
- Orientation = PCA on mask coordinates.
- Centroid from mask (fallback to box centre if mask missing).
- Defect accepted only if mean overlap of defect ROI with bottle mask ≥ 0.30.

4.3 TensorRT Contract

Same runner design as the detector: dedicated stream, shared pinned RGB stage, async H2D / preprocess / inference / D2H, masks decoded and thresholded (sigmoid > 0.5 after resize to frame).

§5 Geometry, OCR, Tracking and Final Decision

5.1 Orientation (Tilt)

Source: bottle segmentation mask (not the box alone). Method: covariance of mask pixel coordinates → principal eigenvector → angle. Limit: ≤ 45° → PASS; > 45° → FAIL. History: angle values stored per track; final value uses median aggregation.

5.2 Horizontal and Vertical Centricity

```
H = (label_x − bottle_x) / bottle_width   |   V = (label_y − bottle_y) / bottle_height
```

Rule	Value
H PASS	|H| ≤ 0.15
V PASS	|V – expected_V| ≤ 0.05
Expected V (100 ml)	0.12
Expected V (300 ml)	0.07
Expected V (500 ml)	0.01

Stabilisation: history window 5, minimum history 3, spatial jump tolerance 0.08. Different expected V values reflect physical label placement on each bottle size.

5.3 Capacity OCR

Item	Recorded behaviour
Engine	PaddleOCR (GPU when available)
Target values	100, 300, 500 only
Min confidence	0.18
Crop policy	Expand capacity box; enhance; optional upscale if short side < 300 px
Frequency	Throttled; initial frame + periodic retry while capacity unknown
Aggregation	Majority / stable capacity over history

OCR failure leaves capacity Pending; it does not by itself force DEFECTIVE.

5.4 Defect Validation

Detection of damage / bump is necessary but not sufficient. Overlap of defect box with bottle mask must be ≥ 0.30. Confirmation requires ≥ 1 valid overlapping frame; 1 missing frame is tolerated before streak reset. Confirmed defects force DEFECTIVE at finalisation (subject to overall rule set).

5.5 Multi-Frame Tracking and Temporal Stabilisation

Match by IoU > 0.40, with centre-distance fallback. Max missing frames: 20. Per-track history stores orientation, H/V, capacity and defect streaks. Final numeric values use median of reliable history; PASS/FAIL tokens use majority. Tracking exists so a single noisy frame does not decide the bottle.

5.6 Finalisation Rules

A track is finalised when either (1) the **trigger line** is crossed (40% of frame width, ±20 px tolerance), or (2) the **missing-frame** limit is reached (20 frames). Best complete / defect frame logic selects the crop saved for audit.

5.7 GOOD / DEFECTIVE / INCOMPLETE Decision

Outcome	Condition
GOOD	Orientation PASS and H PASS and V PASS and no confirmed defects
DEFECTIVE	Any of orientation / H / V is FAIL, or confirmed damage / bump
INCOMPLETE	Any of orientation / H / V still Pending

Pending is not mapped to FAIL.

§6 Unseen / Sample Benchmark and Runtime Evidence

6.1 Sample Operational Results by Capacity

These tables are **sample runs** used for validation observation. They are not a formal sealed test set with precision / recall / F1 against independent ground truth.

Capacity	Total (sample)	Good	Defective
500 ml	16	14	2 (bump)
300 ml	9	6	3 (damage / bump)
100 ml	13	10	3 (bump / damage)
All samples	38	30	8

Detailed per-bottle tilt / H / V / capacity values are retained in results.md / CSV exports.

6.2 Runtime FPS Evidence (sample)

Metric	Scope	Sample value
Average FPS (bottle detected only)	Full pipeline: capture → detection → tracking → OCR → drawing → video → display	~18.66 FPS
Average inference FPS	Preprocess → H2D → TensorRT detection + segmentation → D2H	~74.66 FPS

These figures are observed sample measurements, not a formal A17 target-hardware soak report.

6.3 Optimisation Techniques Applied

- Native TensorRT engines for detection and segmentation
- Dedicated CUDA stream per engine (overlapped execution)
- Shared pinned-host RGB staging buffer
- Asynchronous H2D / D2H; single stream wait per engine
- Reusable device and host output buffers
- GPU mask cache + GPU PCA / centroid / overlap
- OCR on GPU, throttled (not every frame)
- Background image saving via thread pool
- Per-class confidence thresholds tuned for track stability vs defect sensitivity

§7 Traceability and Evidence Chain

7.1 Run and Artifact Register

Record	ID / artifact	Traceability evidence
Dataset	frosch-bottle-5-ypv4gi	Dataset ID recorded; full inventory / hash / sealed split still required
Detector checkpoint	runs/frosch_medium/checkpoint_best_regular.pth	Path referenced in pipeline configuration
Segmenter checkpoint	runs/frosch_seg_medium/checkpoint_best_total.pth	Path referenced in pipeline configuration
Detector engine	output/rfdetr-medium.trt	Runtime load in NativeTRTEngine
Segmenter engine	output/rfdetr-seg-medium.trt	Runtime load in NativeTRTEngine
Detection evaluation	Confusion matrix (IoU ≥ 0.50)	Provided evaluation evidence
Sample operational runs	100 / 300 / 500 ml tables	results.md / CSV
Runtime optimisation	CUDA streams, pinned memory, OCR throttle	optimization.md + code

Formal training run IDs, environment locks, SHA-256 hashes and sealed split manifests are **not** complete in the supplied materials.

7.2 Evidence Flow

A03 Inventory → A04 Schema → A05 QA → A06 Dataset Version → A07 Training → A08 Evaluation
A02 Objective → A09 Measurement Validation
A07 + A08 + A09 + A11 + A12 → A13 Release Manifest → A14 Approval

7.3 Immutable Evidence Rule

Completed training, evaluation and performance records are evidence for the specific runs that created them. A changed model, threshold, split or test environment should generate a **new** run record rather than editing completed evidence in place.

§8 Handbook Compliance and What Must Be Added Now

8.1 A01–A17 Status Matrix

ID	Artifact	Status	Required action
A01	Question Set	Missing	Record signed customer/task answers and assumptions
A02	Objective	Missing	Measurable targets: false GOOD/DEFECTIVE limits, accuracy, line rate, conditions
A03	Dataset Inventory	Partial	Add full counts, capture record, inventory and source hash
A04	Annotation Schema	Missing	Class meanings, mask rules, defect definitions, boundary policy, change log
A05	Annotation QA	Missing	Reviewer, reviewed share, disagreement, edge cases, resolutions
A06	Dataset Version	Partial	Issue sealed version with parent links after A03–A05
A07	Training Run	Partial	Add full config, seed, environment, metrics, hashes for both models
A08	Evaluation	Partial	Add objective-linked criterion table and error analysis
A09	Measurement Validation	Missing	Independent reference for tilt, H/V, capacity, defects; repeatability
A10	Root-Cause Note	Missing	For each validation disagreement requiring investigation
A11	Soak Test	Missing	Sustained line-rate: memory, latency percentiles, recovery
A12	Fallback Audit	Missing	Every failure path explicit; no substituted or stale GOOD
A13	Release Manifest	Partial	Finalise only after parent gates and hashes
A14	Decision Record	Missing	Formal approve/reject, decider, date, alternatives
A15	Re-entry Record	Not supplied	Create only if handbook re-entry trigger occurs
A16	Exception Record	Not supplied	Create only for an approved exception
A17	Model Performance	Partial	Add full target-hardware pipeline benchmark and production comparison

8.2 Priority Documentation Additions

1. Close A01 / A02 first. Without a signed measurable objective, scores cannot be judged pass/fail for release.
2. Formalise A04 / A05. Convert class and geometry conventions into an approved schema and reviewed QA record; then seal A06.

3. Complete evaluation evidence. Tie A08 to A02 criteria; produce independent A09 validation for geometry, OCR and defects.
4. Prove runtime safety. A11 soak / recovery and A12 fallback audit on the live camera path.
5. Finish A17 on target hardware. Full detector + segmenter + OCR + tracking pipeline timing and memory.
6. Release only after manifest and decision. Freeze parents in A13 and record formal A14 approval or rejection.

§9 Final Acceptance Checklist

Release condition	Evidence
■ Signed customer question set and measurable objective	A01, A02
■ Annotation schema approved; QA sample and resolutions complete	A04, A05
■ Dataset version sealed and leak-free with parent links	A06
■ Training runs and selected artifacts traceable by config / hash	A07
■ Unseen evaluation meets the objective for required capacity groups	A08
■ Independent reference / measurement validation passes and is repeatable	A09; A10 where needed
■ Sustained line-rate memory, latency and recovery pass	A11
■ Every failure path is explicit; no substituted or stale result	A12
■ Target-device full-pipeline performance meets budget	A17
■ Final release manifest names every parent and artifact hash	A13
■ Formal approval / rejection decision recorded	A14

9.1 Current Decision

Current status: NOT APPROVED FOR PRODUCTION.
Documented technical work is suitable for continued evaluation and integration. Production release remains blocked by the open acceptance evidence listed above.

9.2 Source Records Consolidated

- Pipeline code and runtime configuration (detection / segmentation TensorRT, OCR, tracking, decision rules)
- README.md — architecture, thresholds, run commands
- camerasetup.md — Vimba X / Harvester setup
- results_saving.md — output files and folders
- results.md — sample operational tables and FPS
- optimization.md — TensorRT / CUDA / OCR / async techniques
- Detection confusion matrix (IoU ≥ 0.50)

9.3 Minimum Fields for the Next Documentation Pass

Artifact	Owner / timing	Minimum information to record
A01 / A02	Engineer + lead, before release	Customer question set; numeric targets; false GOOD/DEFECTIVE limits; line rate; conditions
A04 / A05	Lead + reviewer, before final dataset	Class/defect definitions, mask rules, schema version, reviewed share, change log
A07	Engineer	Full training configs, seeds, environments, metrics, checkpoint and engine hashes
A09	Measurement owner	Independent reference for tilt, H/V, capacity, defects; repeatability; per-capacity results
A11 / A12	Engineer, target runtime	Soak duration, memory, p95 latency, recovery; explicit behaviour on every failure path
A17	Engineer, before release candidate	Selected models, thresholds, target-device timing, throughput, memory, production comparison
A13 / A14	Lead / final decider	Parent IDs, hashes, approval state, decision, date, decider, rejected alternatives

9.4 Release Control Block

Control field	Value / sign-off
Document version	1.0 — consolidated evaluation handoff
Current release state	NOT APPROVED FOR PRODUCTION
A13 release manifest ID	________________
A14 decision record ID	________________
Technical review / date	________________
AI Lead approval / date	________________
Production comparison reference	________________

Complete this control block only after A01/A02, A04/A05, final A06, objective-linked A08/A09, A11, A12 and target-device A17 evidence are closed. The signed A13/A14 records are the release decision, not this technical handoff by itself.